

NLP Fundamentals

Sentiment, intent, summarisation, translation, moderation — every product team needs to read text at scale. Today is the foundation under every one of those features.

Is "Bahut bakwaas" spam, praise, or criticism? Text is messy. Pipelines make it production-ready.

WHAT'S INSIDE

- 01** The NLP pipeline
- 02** TF-IDF — "rare but repeated"
- 03** Sparse (TF-IDF) vs Dense (Embeddings)
- 04** Three tasks every team ships
- 05** Spam classifier — 12 lines

WHY THIS LESSON MATTERS

India posts ~2 billion Hinglish tweets a year.

WHY

Real

A real reason this matters in industry today.

SCALE

Today

A real reason this matters in industry today.

SPEED

Now

A real reason this matters in industry today.

IMPACT

Yours

A real reason this matters in industry today.

LEARNING OUTCOMES

By the end of this lesson, you will be able to —

1

Tokenise, lowercase, and strip stopwords from raw text.

2

Explain TF-IDF and what it represents.

3

Distinguish sparse vectors from dense embeddings.

4

Describe 3 NLP tasks and their metrics.

5

Build a spam classifier in 12 lines of scikit-learn.

AGENDA

What we'll cover today

Short, sequenced parts. Each one builds on the last.

01

The NLP pipeline

5 deterministic steps

02

TF-IDF — "rare but repeated"

Weights discriminating words

03

Sparse (TF-IDF) vs Dense...

TF-IDF is your first model.
Embeddings are your production system.

04

Three tasks every team ships

Classify · Extract · Summarise

01

PART 1 OF 4

The NLP pipeline

5 deterministic steps

CONCEPT 1 OF 4

The NLP pipeline

1. Normalise — lowercase, strip punctuation, unicode-clean.
2. Tokenise — split to words or subwords.
3. Remove stopwords (task-dependent).
4. Stem / lemmatise (optional).
5. Vectorise — convert to numbers.

SPINE

Normalise → Vectorise

Same spine from 1995 to 2026. Only the vectoriser changed.

DEFINITION

The NLP pipeline

5 deterministic steps

EXAMPLE

Real-world example: applies directly to Indian industry contexts.

CONCEPT 2 OF 4

TF-IDF — "rare but repeated"

TF — how often the word appears in THIS document.

IDF — how rare the word is across the corpus.

TF-IDF = $TF \times IDF$ — high when common-here AND rare-elsewhere.

Ruthlessly effective for classification on modest data.

INTUITION

Rare + Repeated

"Return" in a shopping email is TF-IDF gold.

KEY POINTS

- TF — how often the word appears in THIS document
- IDF — how rare the word is across the corpus
- $TF-IDF = TF \times IDF$ — high when common-here AND rare-elsewhere
- Ruthlessly effective for classification on modest data

CONCEPT 3 OF 4

Sparse (TF-IDF) vs Dense (Embeddings)

TF-IDF is your first model. Embeddings are your production system.

DEFINITION

Sparse (TF-IDF) vs Dense (Embeddings)

TF-IDF is your first model. Embeddings are your production system.

EXAMPLE

Real-world example: applies directly to Indian industry contexts.

CONCEPT 4 OF 4

Three tasks every team ships

Classification — one label per document. Metric: F1.

NER (Named Entity Recognition) — extract people, places, dates. Metric: token F1.

Summarisation — shorter version preserving meaning. Metric: ROUGE.

These three cover most industry NLP value.

COVERAGE

3 tasks · 3 metrics

Know these; you can scope 80% of NLP work.

KEY POINTS

- Classification — one label per document. Metric: F1
- NER (Named Entity Recognition) — extract people, places, dates. Metric: token F1
- Summarisation — shorter version preserving meaning. Metric: ROUGE
- These three cover most industry NLP value

02

PART 2 OF 4

Engage and reflect

Pause, talk, and connect what you just learned to your own work.

✦ THINK • PAIR •
SHARE

ACTIVITY

Clean a Hinglish tweet end-to-end

PROMPT

Apply the four concepts above to one real example from your own week.

HOW TO RUN IT

- 1
- 2 Normalise, tokenise, remove stopwords. Compare with partner.
- 3 One pair types their 5-step pipeline on the instructor's screen.
- 4 Reveal: instructor confirms the canonical answer.

REFLECT & APPLY

Three questions before you close this lesson

Spend 60 seconds on each. Write the answers down — no scrolling, no shortcuts.

Q1

Which of the 3 NLP tasks excites you most — and why?

Q2

For YOUR dataset, would TF-IDF or embeddings be a better first model?

Q3

Where might stopwords removal actually hurt performance?

03

PART 3 OF 4

Knowledge check

Three short questions. One minute each. Honest answers only.

TF-IDF assigns high weight to:

A

Common words

B

Rare words in corpus

C

Long words

D

Uppercase words

TF-IDF assigns high weight to:



CORRECT ANSWER

Rare words in corpus

WHY

Words that appear often in this doc but rarely elsewhere — the discriminators

Typical OpenAI ada-002 embedding dimension:

A

B

B

D

C

Answer: C. text-embedding-ada-002 outputs 1536-dim vectors.

D

(no option)

Typical OpenAI ada-002 embedding dimension:



CORRECT ANSWER

D

WHY

The correct option matches the lesson's core principle.

Metric for NER:

A

MAE

B

Token F1

C

ROC-AUC

D

BLEU

Metric for NER:



CORRECT ANSWER

Token F1

WHY

NER is evaluated at token level — precision and recall per token, then F1

04

PART 4 OF 4

Apply and close

One thing to remember. One thing to ship this week.

SUMMARY

Five sentences to remember

If you remember nothing else from this lesson, remember these five lines.

- 1 Pick the right tool — never the loudest one.
- 2 Practice the framework on a real example.
- 3 Watch for the three classic pitfalls.
- 4 Document your decision in one sentence.
- 5 Carry the discipline forward to the next lesson.

WORKED EXAMPLE

How this lesson plays out in one real Indian context

THE SCENARIO

1. Normalise — lowercase, strip punctuation, unicode-clean.
2. Tokenise — split to words or subwords.
3. Remove stopwords (task-dependent).
4. Stem / lemmatise (optional).
5. Vectorise — convert to numbers.

SPINE

Normalise → Vectorise

Same spine from 1995 to 2026. Only the vectoriser changed.

THE TAKEAWAY

"Normalise → tokenise → vectorise → classify. Same spine for 30 years. Only the vectoriser changed."

WHAT TO NOTICE

1

The NLP pipeline

2

TF-IDF — "rare but repeated"

3

Sparse (TF-IDF) vs Dense...

4

Three tasks every team ships

Mini assignment — Kaggle SMS Spam

Build a spam / ham classifier on the Kaggle SMS Spam Collection.

DELIVERABLES

- Load the dataset; split 80/20.
- Build the 12-line pipeline.
- Report macro-F1.
- Analyse 3 misclassified examples by hand.

WHAT 'GOOD' LOOKS LIKE

Deliverable: Notebook URL + macro-F1 + 3-line error analysis on the forum.

ONE THING TO REMEMBER

“

"Normalise → tokenise → vectorise → classify. Same spine for 30 years. Only the vectoriser changed."

— AI Learning Hub

END OF LESSON 3

What you can now do

Each one of these is now part of your working vocabulary.

- 1 Pick the right tool — never the loudest one.
- 2 Practice the framework on a real example.
- 3 Watch for the three classic pitfalls.
- 4 Document your decision in one sentence.
- 5 Carry the discipline forward to the next lesson.

NEXT → Lesson 5 — Backend with Node.js & REST. The other rail starts here.